

道に沿って物を並べる — N

だけでは坂で杭が倒れる。up を足せば 0度。Pack で 250倍速い

2026-09-19

何を作ったか

S字の道（曲線）に沿って、両側に杭を並べた。手順ページに書く前に、間隔・向き・重さの3つを測った。

間隔。resample の Length に 1.0 を入れても、点の間隔はちょうど 1.0 にはならなかった。長さ 25.02 の道が27点・26区間に切られ、間隔は約 0.96 ($25.02 \div 26 = 0.962$)。Length は「これより長くしない」という上限で、全体をそれ以下の長さで等分する。最後の区間だけ短くなることもなく、Maintain Last Vertex の入り・切りでも変わらなかった。

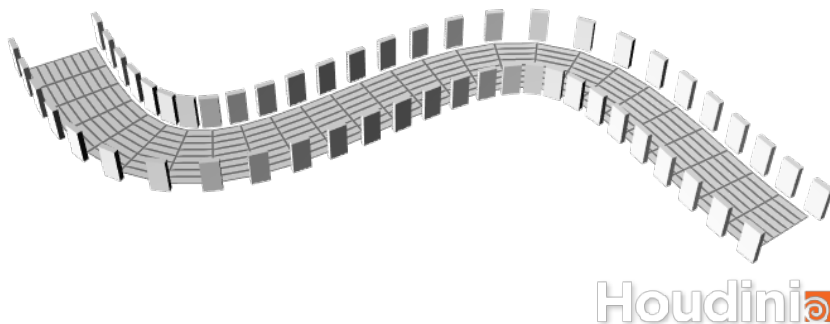
向き。resample で道の向き (tangentu) を点に付け、それを N に入れて copytopoints で配ると、杭の前 (+Z) は道の向きとずれ 0.0000度で一致した。N を入れなければ、杭はすべて同じ向きのままで、道とのずれは平均 84度。

平らな道では、N だけでも杭は傾かなかった。ところが道に起伏（高さ ± 1.2 ）を付けると、N だけでは杭が横に傾き、最大 49度・平均 19度。N と一緒に up に (0, 1, 0) を入れると、傾きは 0.0000度になった。

重さ。細かい球 (48×48) を配ると、Pack and Instance の差が大きい。1万個で、切りだと 2,210万点・1,878MB・714ms、入りだと 1万点・4MB・2.9ms。約250倍速く、メモリは約460分の1。

完成した道

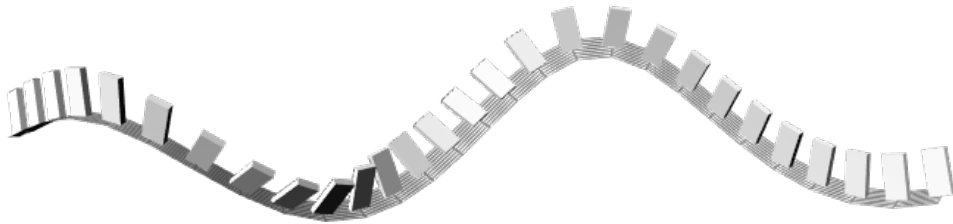
長さ 20 の直線を $z = 3 \sin(0.35 x)$ で曲げ、resample (Length 1.0) で切った。各点を道の横向き (N $\times$ up) $\times \pm 1.2$ ずらして2本にし、 $0.15 \times 1.0 \times 0.5$ の杭を配った。道の面は sweep の Ribbon (幅 2)。



54本の杭が、道の両側で道の向きにそろって立っている。

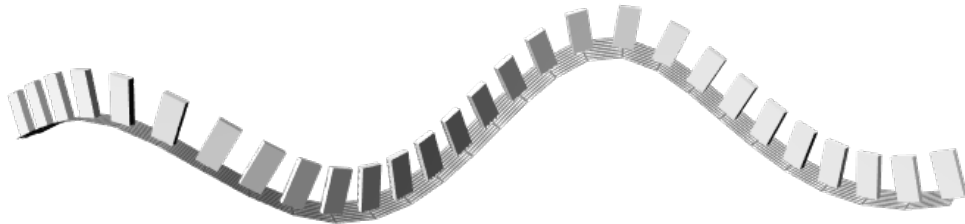
B. 起伏のある道での杭の向き

上が N だけ、下が N と up。道の高さは ± 1.2 で上下する。2枚とも同じカメラ。



Houdini

N だけ。谷のあたりの杭が横に倒れている（最大 49度）。



Houdini

N と up。坂に合わせて前後には傾くが、横には倒れない（0.0000度）。

道	向きの入れ方	前と道の向きのずれ（最大）	横の傾き（最大）	横の傾き（平均）
平ら	N と up	0.0000度	0.0000度	0.0000度
平ら	N だけ	0.0000度	0.0000度	0.0000度
平ら	入れない	135.95度	0.0000度	0.0000度
起伏あり	N と up	0.0000度	0.0000度	0.0000度
起伏あり	N だけ	0.0000度	48.96度	19.02度
起伏あり	入れない	134.83度	0.0000度	0.0000度

A. resample の Length 1.0 で切った間隔

間隔は点どうしの直線距離。道は曲がっているの、道に沿った長さ（0.962）より少し短い。

Maintain Last Vertex	道の長さ	点の数	間隔（最後以外）	最後の間隔
切り	25.0217	27	0.9576～0.9624	0.9621

Maintain Last Vertex	道の長さ	点の数	間隔（最後以外）	最後の間隔
入り	25.0217	27	0.9576~0.9624	0.9621

C. 細かい球（48 × 48・2,210点）を配る

毎回まっさらに組み直し、配る直前までを計算させてから、copytopointsの最初の計算を3回測った中央値。

配る数	Pack and Instance	計算の時間	点の数	メモリ
1,024	切り	75.4 ms	2,263,040	192.3 MB
1,024	入り	0.3 ms	1,024	0.4 MB
10,000	切り	713.8 ms	22,100,000	1,878.0 MB
10,000	入り	2.9 ms	10,000	4.1 MB

わかったこと

- ・ resample の Length は「上限」で、全体を等分する。1.0 を入れると、長さ 25.02 の道が26等分されて間隔は約 0.96。ちょうど 1.0 にしたいなら、道の長さのほうを合わせる。最後の区間だけ短くなることはなかった。
- ・ N に道の向きを入れれば、前はぴったり道を向く。resample の Create Tangent Point Attribute で tangentu を作り、@N = normalize(v@tangentu)。ずれは 0.0000度。copytopoints は形の +Z を N に合わせる（実験008）。
- ・ 坂のある道では、N だけでは杭が横に倒れる。高さ ±1.2 の起伏で、横の傾きが最大 49度・平均 19度。up に (0, 1, 0) を足すと 0.0000度。平らな道では N だけでも 0度だったので、平らなところで確かめただけでは気づけない。
- ・ 道の横向きは N × up で作れる。cross(@N, v@up) を正規化して ±1.2 ずらすと、道の両側に並ぶ。
- ・ 細かい形をたくさん配るなら、Pack and Instance を入れる。1万個で 714ms → 2.9ms（約250倍）、1,878MB → 4.1MB（約460分の1）。点の数は 2,210万 → 1万。
- ・ 測り方で2回つまずいた。1つ目。配る先の板をわずかに動かして計算し直させると、Pack 切りでも 9.5ms と出た。位置だけが変わったときは、全部を作り直してないらしい（確かめてはいない）。毎回組み直して測ると 714ms だった。2つ目。点の数を len(g.points()) で数えると、2,210万点を Python の一覧にする時間（約8.8秒）が乗った。intrinsicValue("pointcount") で数えれば乗らない（実験077と同じ話）。
- ・ 計算の時間は、道・杭54本・道の面208枚で 0.097秒。

次にやること

- ・ 配る物に大きさや向きのばらつきを付ける（pscale・orient）
- ・ 曲線の曲がり具合に合わせて、間隔を変える
- ・ 振り返り点検（061~090）を 090 で行う